

Learning Objectives

Learners will be able to...

- Describe pre-trained language models and how to use them
- Create an open domain chatbot using a pre-trained model from the transformers library, with a tkinter GUI

info

Make Sure to Know

Intermediate Python.

Limitations

In this assignment, we only work with the tkinter and transformers libraries.

Connecting to Github

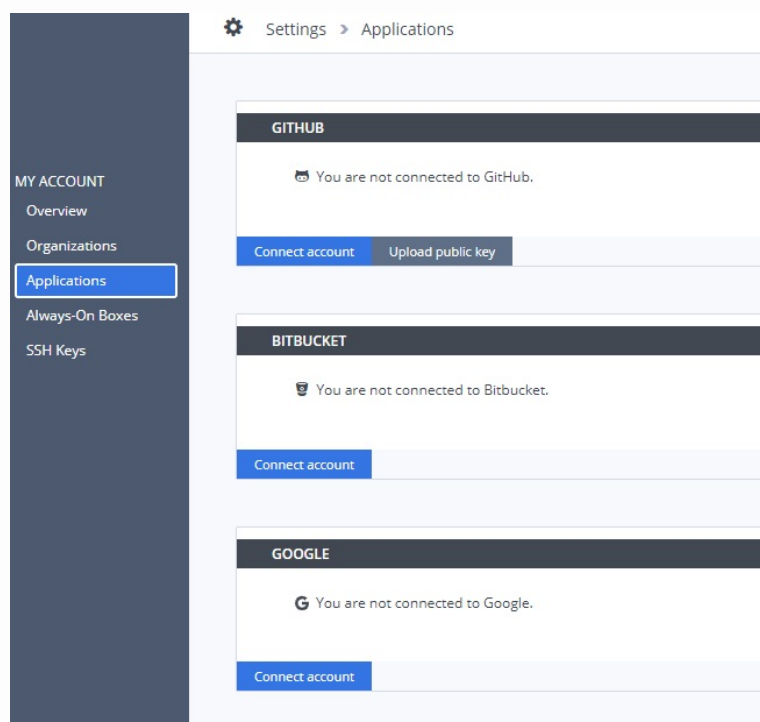
Connecting to GitHub

In this lab, you are going to build a Chatbot that uses a pre-trained model. In order to showcase this project for your portfolio, you are going to use GitHub. If you do not yet have an account, please [create one](#) now. We are going to clone a repo that will contain the code for your Chatbot.

Connecting GitHub and Codio

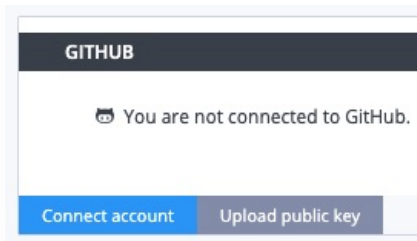
You need to connect GitHub to your Codio account. This only needs to be done one time.

- In your Codio account, click on your username
- Click on **Applications**



Codio Account

- Under GitHub, click on **Connect account**



connect account

- You will be using an SSH connection, so you need to click on **Upload public key**

Fork the Repo

- Go to the [nlp-pretrained-model-bot](#) repo. This repo is the starting point for your project.
- Click on the “Fork” button in the top-right corner.
- Click the green “Code” button.
- Copy the **SSH** information. It should look something like this:

```
git@github.com:<your_github_username>/nlp-pretrained-model-bot.git
```

info

Important

If you do not use the SSH information, you will have to provide your username and Personal Access Token (PAT) to GitHub each time you push or pull from the repository. See this [documentation](#) for setting up a PAT for your GitHub account.

In the Terminal

- Clone the repo. Your command should look something like this:

```
git clone git@github.com:<your_github_username>/nlp-pretrained-model-bot.git
```

- You should see a `nlp-pretrained-model-bot` directory appear in the file tree.

On the last page, we'll show you how to push to Github.

You are now ready for the lab!

Recap: What are pre-trained language models?

- A language model is a model which understands language – more precisely how words occur together in natural language.
- A language model is used to predict what word comes next.
- The building and training of models is both complex and resource intensive. Luckily, there are pre-trained language models.
- A couple of factors to consider when choosing a pre-trained language model:
 1. What task are you using it for?
 2. What are the technical requirements to use the model?

The Hugging Face transformers package

Hugging Face is an AI community and Machine Learning platform that aims to making NLP and Machine Learning accessible to everyone.

Specifically, their transformers package provides APIs and tools to easily download and train over 20,000 state-of-the-art pretrained models.

These models support common tasks in a variety of use cases, such as:

- Natural Language Processing
- Computer Vision
- Speech Recognition and Audio Classification

...

info

Why should I use a pretrained model ?

Using pretrained models can reduce your compute costs, carbon footprint, and save you the time and resources required to train a model from scratch.

GUI for the chatbot

info

chatbot_gui.py

We'll be using the same functions we used in The Simple Bot and the API Bot for the tkinter GUI.

These functions can be found in the `chatbot_gui.py` file, in the top left pane.

To recap:

- `baseWindow` -
- `chatWindow` -
- `userEntryBox` -
- `sendButton` -
- `send()` -
- `create_and_insert_user_frame()` -
- `create_and_insert_bot_frame()` -

Blenderbot by Facebook (now Meta)

The pretrained model we're using in this lab, is the BlenderbotSmall model by Facebook (now Meta).

The special things about this model are:

- It simultaneously builds its own long-term memory that it can continually access
- It can search the internet for timely and up-to-date information
- It can have conversations on nearly any topic: a great recipe for an open-domain chatbot!
- It remembers and separates the data for each person it speaks with, and can leverage this data to have conversations that may continue for days, weeks, or even months!

The `get_bot_response` function using a pre-trained model

info

Writing Code

The transformers package has already been installed in your project.

To your left, you should see an opened file called `pretrained_model.py`. Use this file and reference the syntax below to work on this Lab.

In order to build our sophisticated chatbot, we start by importing the necessary libraries in the `pretrained_model.py` file.

The next step is to write a function that initializes the pretrained model, and the `get_bot_response` function that runs the `user_input` through it and returns a response.

Paste the following code in the `pretrained_model.py` file, in the top left pane:

```
import os
import transformers

def initialize_model():

    model = transformers.pipeline("conversational",
                                  model="facebook/blenderbot_small-90M")
    os.environ["TOKENIZERS_PARALLELISM"] = "true"

    return model

def get_bot_response(model, user_input):

    chat = model(transformers.Conversation(user_input))
    bot_response = str(chat)
    bot_response = bot_response[bot_response.find("bot >>")
                                +6:].strip()

    return bot_response
```

And that's it! We've successfully implemented the logic for our chatbot to use a pretrained model! Let's run it and interact with it!

info

Note

The bot takes a few seconds to process each user input and respond. So don't worry if it looks like it's frozen, it's just thinking!

Pushing to Github

Pushing to GitHub

To push your work to GitHub, do the following in the terminal:

- Commit your changes:

```
git add .  
git commit -m "Finished The Pretrained Model Bot"
```

- Push to GitHub:

```
git push
```